

----- Moving MongoDB to the Cloud

Vinicius M. Grippa

Senior Support Engineer for MySQL/MongoDB

vinicius.grippa@percona.com

About me

- Support Engineer at Percona since 2017.
- Working with MySQL for over six years.
- Working with databases for over nine years.
- Speaker at Percona Live conferences and meetups about MySQL/MongoDB.
- Social media
 - <https://www.linkedin.com/in/vinicius-grippa/>
 - vinicius.grippa@percona.com / vgrippa@gmail.com

About Percona

- Founded in 2006
- 190+ staff worldwide
- Percona blog has **100k+ readers** per month
- Percona tools have been used **over a billion times** in the past 3 years
- Percona conferences attract **thousands of attendees** from some of the world's largest companies (**#PerconaLive**)
- Percona software is installed on **millions of servers**

MongoDB in the Cloud

MongoDB Hosting Options

A few MongoDB host options:

- On-premises data centers
- Cloud providers such as AWS, Azure, and others
- Hosted Databases (ObjectRocket, Rackspace)
- Hybrid of on-premises data center and cloud providers

Reasons to Move To Cloud

- Cloud in general is cost-effective
- No need to employ additional staff to manage your hardware and configurations — The Provider will take care of that.
- No need to pay for the extra effort to implement security on your DB application.
- Cloud data centers are geographically dispersed, which ensures high availability by distributing MongoDB replica set members across different zones and regions.
- With automated backup and disaster recovery, the impact of failures can be mitigated with reduced downtime and high availability. This can easily be achieved at a low cost.

Strategy Planning

- The type of the instance (CPU, Memory, Disk (EBS and Ephemeral))
- High availability strategy (Regions, AZ, Geo-Distributed)
- Security design, including encryption, SSL, VPN, networking, firewalls, and security groups for the VPCs and MongoDB (certificates, bind, authentication)
- Migration / Rollback strategy
- Backup and recovery strategy

Strategies to Migrate

Strategy #1 - Adding AWS MongoDB Server as a Replica Set Member - No Downtime Required

- Perform connectivity check.
- On AWS, start MongoDB with the same configuration as your on-premises data center MongoDB configuration.
- Add the AWS server into your own data center replica set using **rs.add()** and wait until initial sync completes.
- Once the initial sync is completed, you can perform a **rs.stepdown()** and evaluate the performance. Once the tests are concluded, remove the member from the on-premises data center using **rs.remove()**.

Strategy #2 - Backup and Restore to AWS — Downtime is Required

- Take backup (Percona HotBackup, PBM) or mongodump off the database from the on-premises data center.
- Install and configure brand new MongoDB on AWS, and restore the database to AWS. Step 3: Direct your application to AWS.

Attention Points

What do we need to consider?

- Latency
- Downtime
- Security
- Performance

Latency

- Read Concern - *control the consistency and isolation properties of the data read from replica sets and replica set shards.*
- Write Concern - *describes the level of acknowledgment requested from MongoDB for write operations to a standalone mongod or to replica sets or to sharded clusters.*
- Config Servers (if sharded) - *When writing to the config servers, MongoDB uses a write concern of "majority".*
- Application Location and configuration string

Downtime

It is important to have defined the backup/rollback strategy otherwise the downtime can be even higher. Especially when using shards and it is common to have terabytes of data involved.

What do we need to consider?

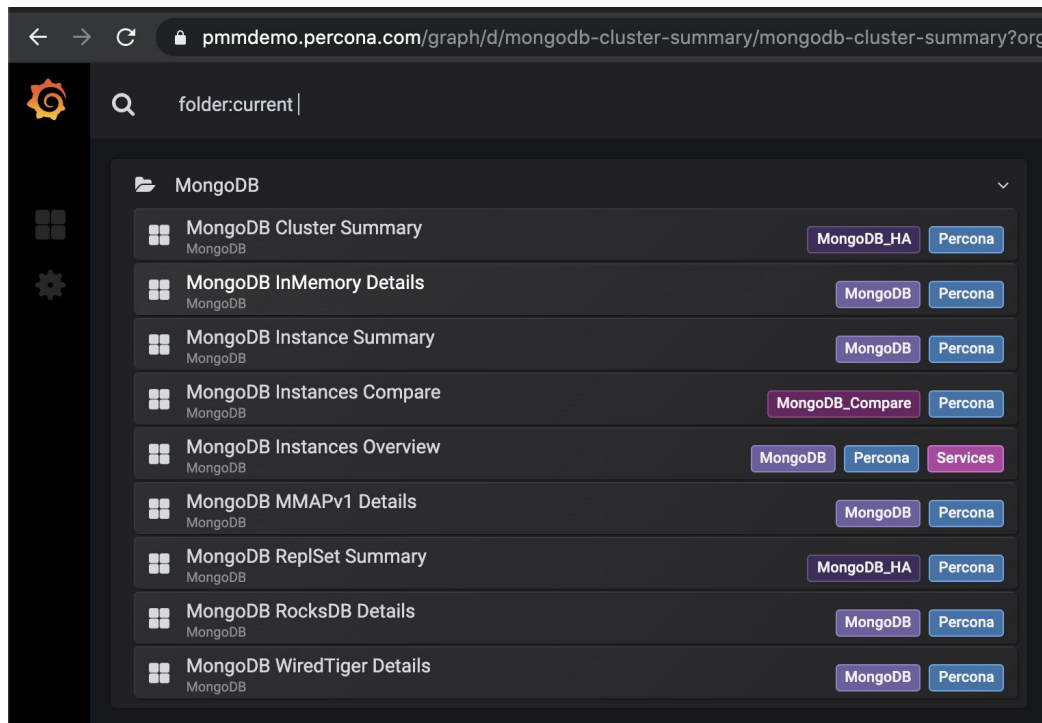
Security is extremely important!

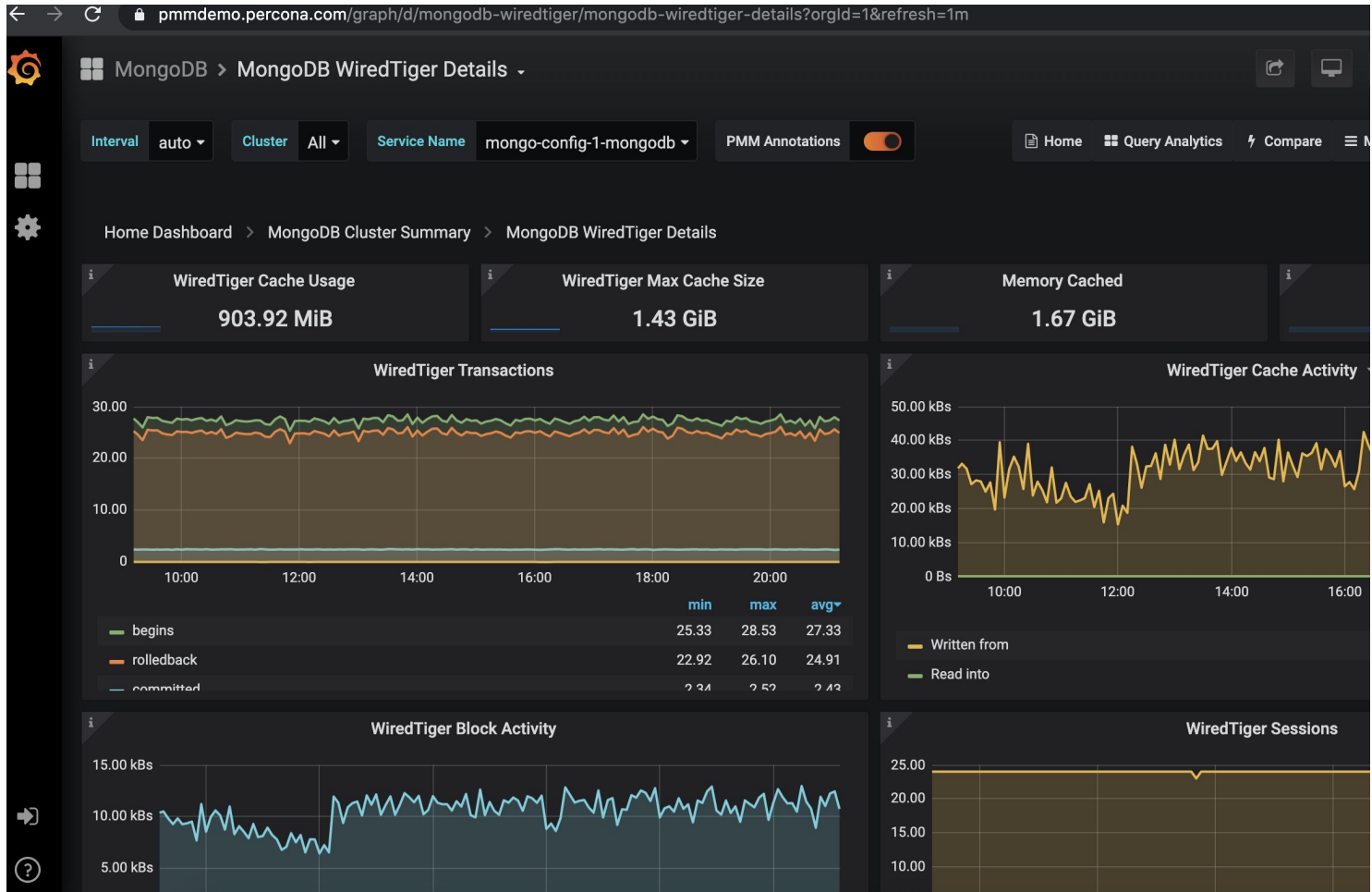
Consider setting the server behind a private network.

Don't expose MongoDB port (if needed change the default port, use authentication, SSL, certificates X.509, and roles)

What do we need to consider?

Monitor your environment!





What we should NOT do?

What we should not do?

- Upgrade/Downgrade MongoDB version
- Upgrade/Downgrade Driver version
- Adjust MongoDB/OS parameters

DEMO

Kubernetes Deployment using Percona MongoDB Operator

Percona Operator for MongoDB - Advantages

Operators help in building cloud-native applications by delivering automation advantages like deploying, scaling, and backup and restore while being able to run anywhere Kubernetes is deployed.

- Deploy easily - create a Percona Server for MongoDB environment with no single point of failure and the ability to span multiple activity or availability zones
- Scale Your Replica Set – change the size of the replica set by altering the size key in the Custom Resource options configuration
- Manage your Backups – automate backups or perform an on-demand backup at any time and support simple restores
- Set Node as Arbiter – set a node as an arbiter, which participates in elections for a new primary node but does not store any data
- Automate node recovery - use the self healing capability to automatically recover from failure of a single Percona Server for MongoDB node
- Provide data encryption - rely on support for data encryption in transit

DEMO



PERCONA



BLOG Registration

<https://learn.percona.com/database-performance-blog-subscription>



WEBINARS

<https://www.percona.com/resources/webinars>



EVENTS

<https://www.percona.com/community/events>

Look for us:



谢谢

Thank you

Grazie

Obrigado

Gracias

Open Source Database Experts

Open Source Database Experts